STRUCTURE

It is a user deSfined data type.

It is a complex data type.

It is collection of heterogeneous variables.

Structure is a user defined, complex data type where we can store and manage more than one variable of different data types under one name.

Structure allows to store both primitive and derived data types (arrays, pointers) at one place, under one name.

In real time applications, data is stored in the form of objects. In this situation,

we need structures. Structures are the foundation for object oriented.

Primitive and derived data types are designed to work with basic data types like int, float and char.

Primitive and derived data types don’t support real time requirements. Hence we have to use the user defined data type structure.

Structure allows to carry different types of variables at a time.

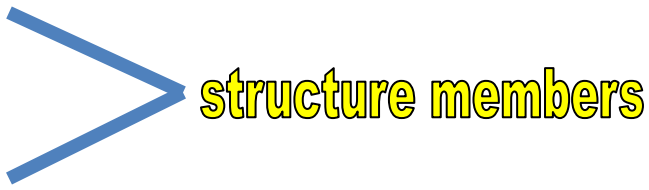
When structure address is available, automatically all the variables address also available. Due to this search time is reduced.

Structure allows to store information in the form of records.

In data files we are using structures very much.

Syntax:

```
struct   [ <structure-tag-name> ]
{
datatype    variable;
datatype    variable;
}
```



[structure_variables] ;

Here struct is a keyword.

Structure tag name is used to identify the structure and it is optional, but required when structure variables declared in other places of the program.

The variables that are declared inside a structure are called **structure members**.

Structure size is sum of all the structure members datatype size.

Without structure members [empty] structure size is **1 byte.**

Structure variables are the **instances** [**copies**] of the structure.

Structure is a blue-print [original copy] to create the structure variables.

Structure variable is the physical representation of a structure.

When structure variables declared then only memory allocated for structure members.

Every structure should be end with ;

To access the structure members we should have to use the following syntax.

```
structurevariable.structuremember;
```

It is called calling / accessing / invoking the structure members.

Here . (dot) operator is called

- Member access operator
- Field access operator
- Member of operator
- Membership operator
- Belongs to operator

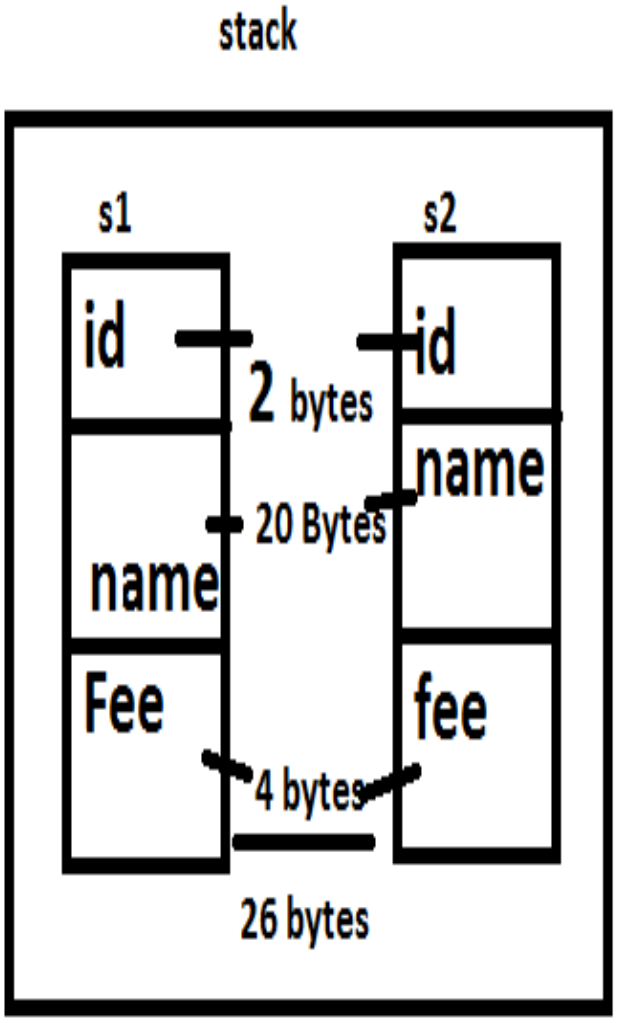
We can declare structure variables in other places of the program by using below syntax.

```
struct   structure-tag-name   structure-variables;
```

Eg: struct stu s1, s2;

Memory allocation for structure variables

```
struct  stu ——— stru tag name
{
int id;
char name[20];        } stru members
float fee;
}
s1, s2; ——— stru variables
```



s1.id=100;/* calling structure member*/

s2.id=200;

Eg: Direct initialization of structure members:

It is the process of passing values for structure members, without using scanf() at design time using =.

Note: In direct initialization of structure members, the passing values datatype and structure members datatype should be matched.

When all the structure members are not initialized, they will store the default values as follows.

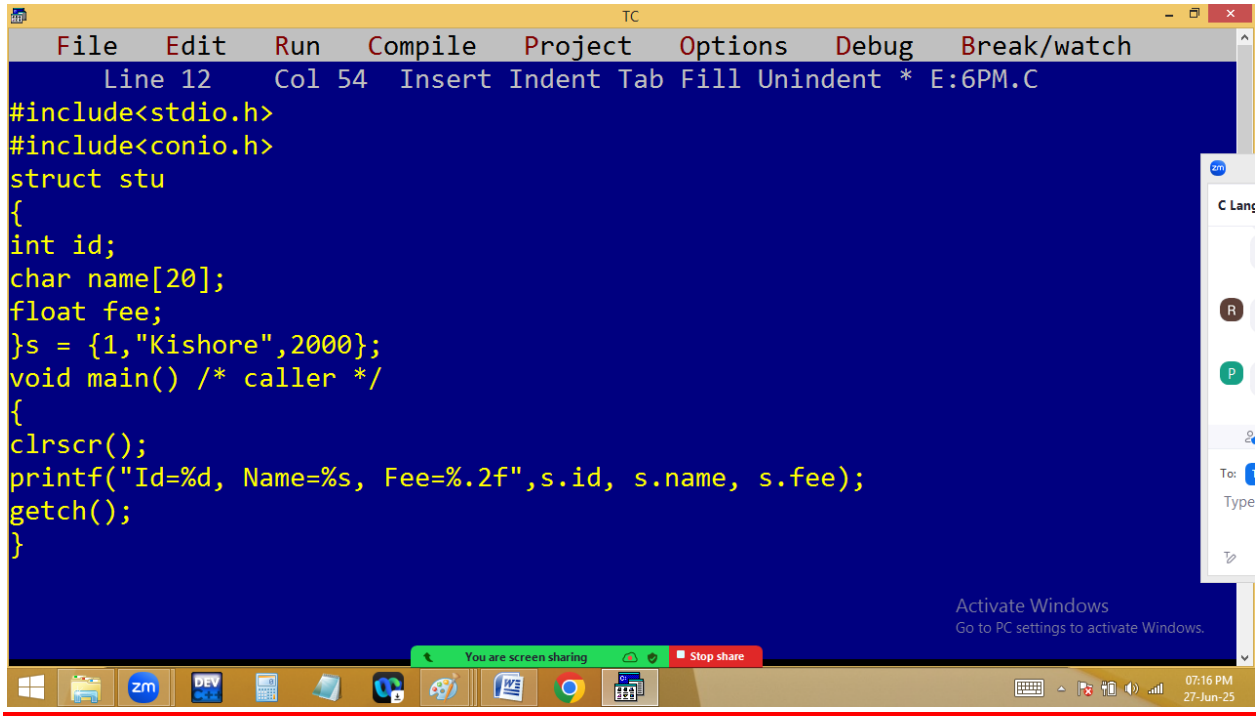
Int – 0

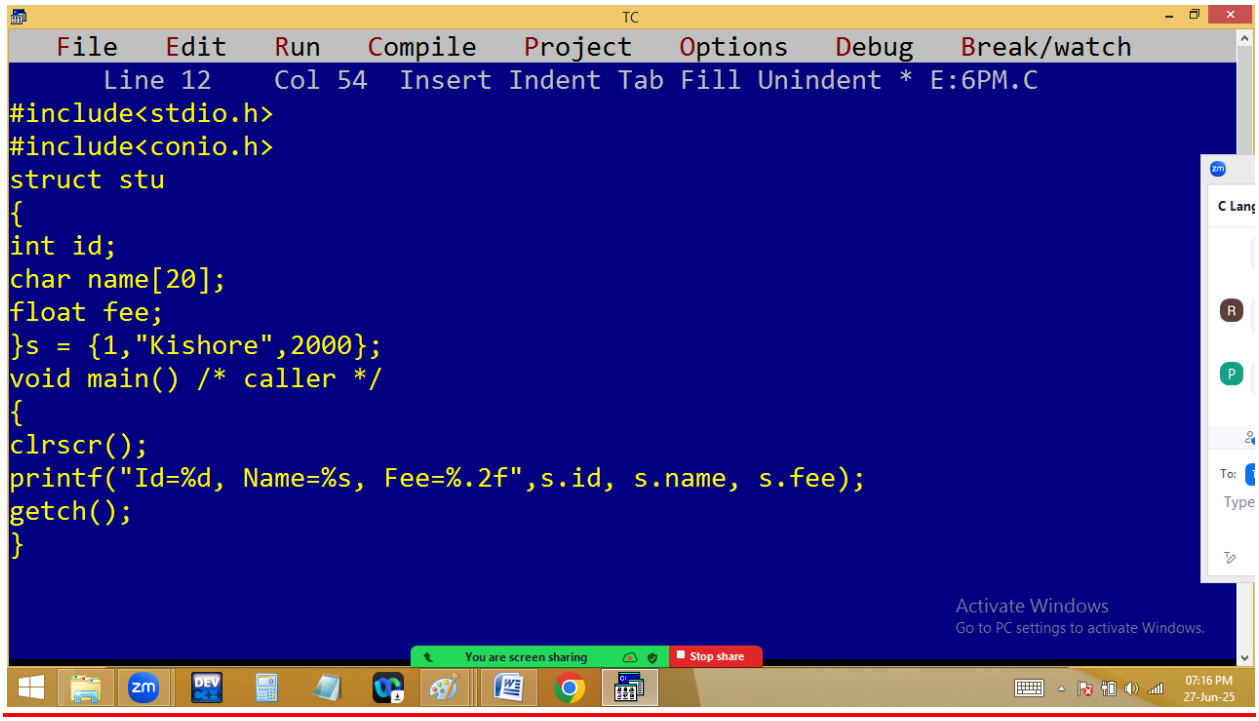
Float – 0.000000

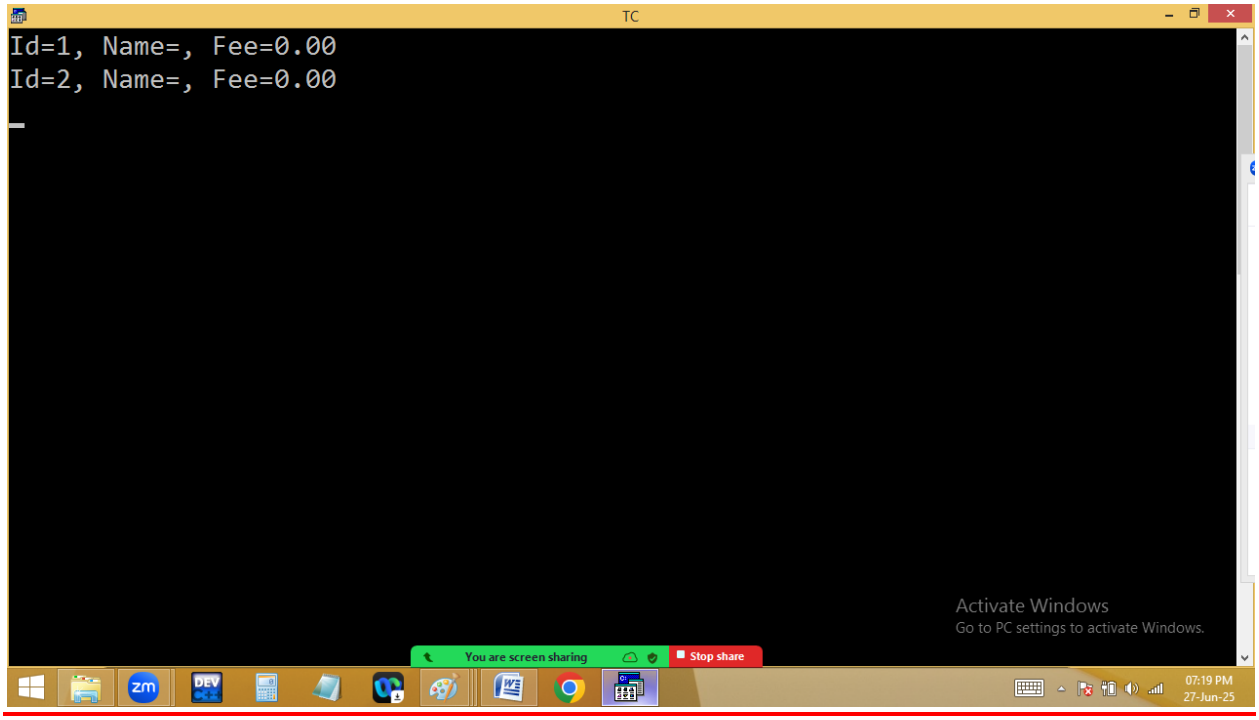
Char – blank space

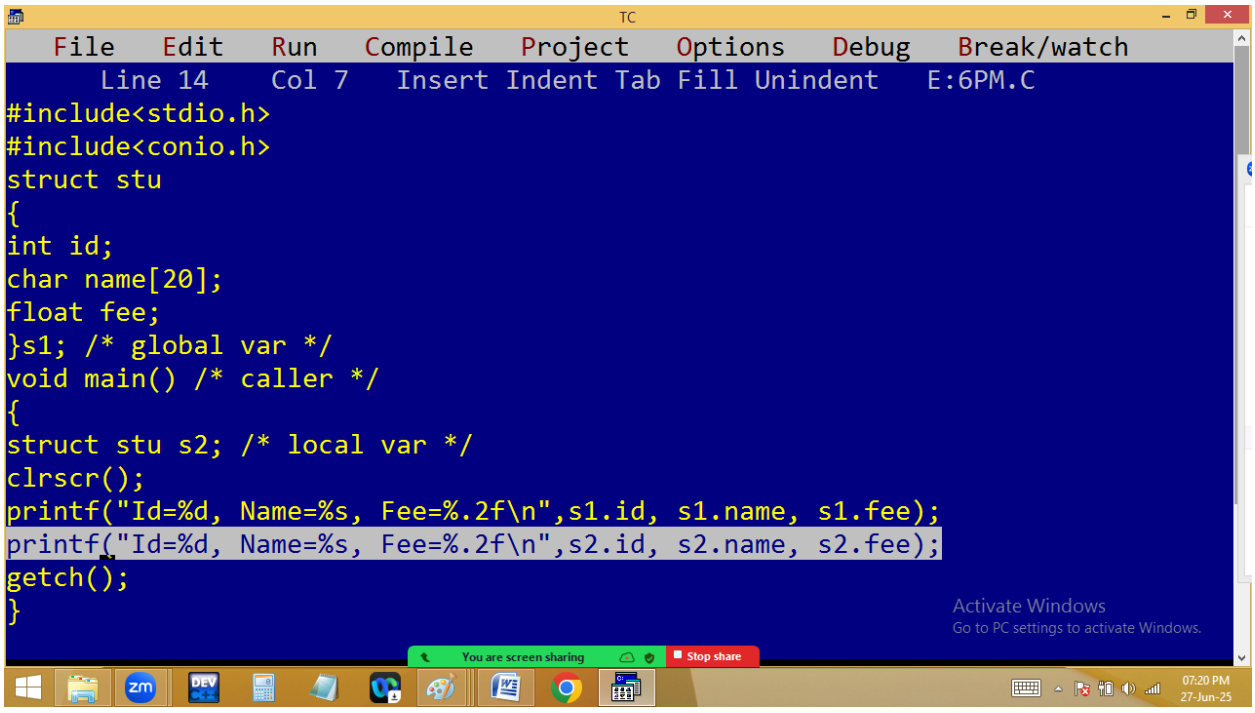
structures stores 0, 0.00 and blank in int, float and char.

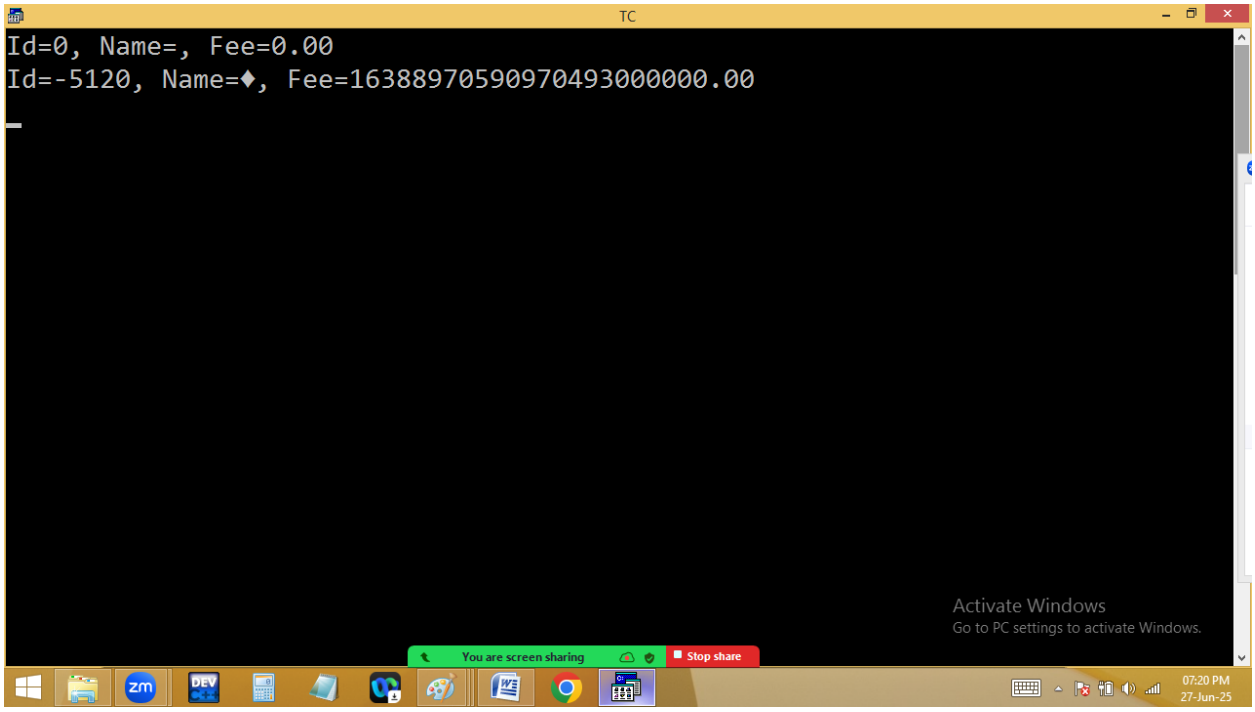
when it is a local structure, without initialization, stores garbage values.



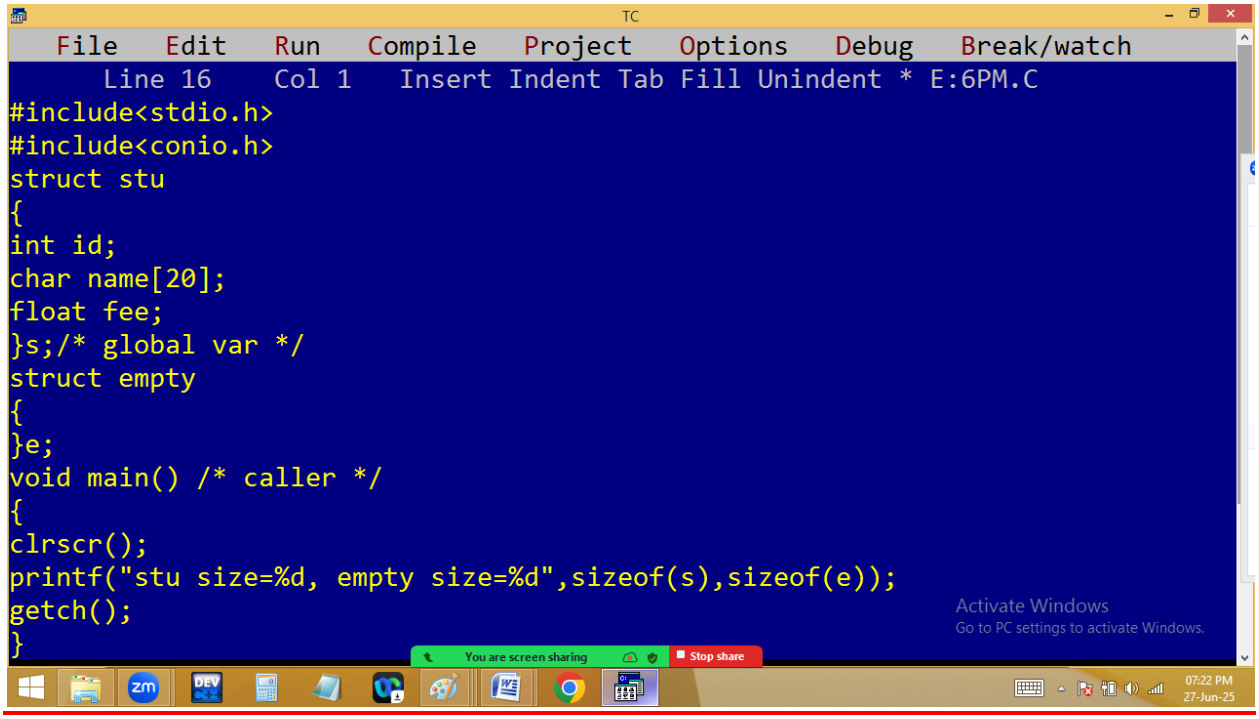

TC
File Edit Run Compile Project Options Debug Break/watch
Line 12 Col 54 Insert Indent Tab Fill Unindent * E:6PM.C
#include<stdio.h>
#include<conio.h>
struct stu
{
int id;
char name[20];
float fee;
}s = {1,"Kishore",2000};
void main() /* caller */
{
clrscr();
printf("Id=%d, Name=%s, Fee=%.2f",s.id, s.name, s.fee);
getch();
}
Activate Windows
Go to PC settings to activate Windows.
You are screen sharing
Stop share
07:16 PM
27-Jun-25


TC
Id=1, Name=, Fee=0.00
Id=2, Name=, Fee=0.00
Activate Windows
Go to PC settings to activate Windows.
You are screen sharing
Stop share
07:19 PM
27-Jun-25


TC
File Edit Run Compile Project Options Debug Break/watch
Line 14 Col 7 Insert Indent Tab Fill Unindent E:6PM.C
#include<stdio.h>
#include<conio.h>
struct stu
{
int id;
char name[20];
float fee;
}s1; /* global var */
void main() /* caller */
{
struct stu s2; /* local var */
clrscr();
printf("Id=%d, Name=%s, Fee=%.2f\n",s1.id, s1.name, s1.fee);
printf("Id=%d, Name=%s, Fee=%.2f\n",s2.id, s2.name, s2.fee);
getch();
}
Activate Windows
Go to PC settings to activate Windows.
You are screen sharing
Stop share
07:20 PM
27-Jun-25

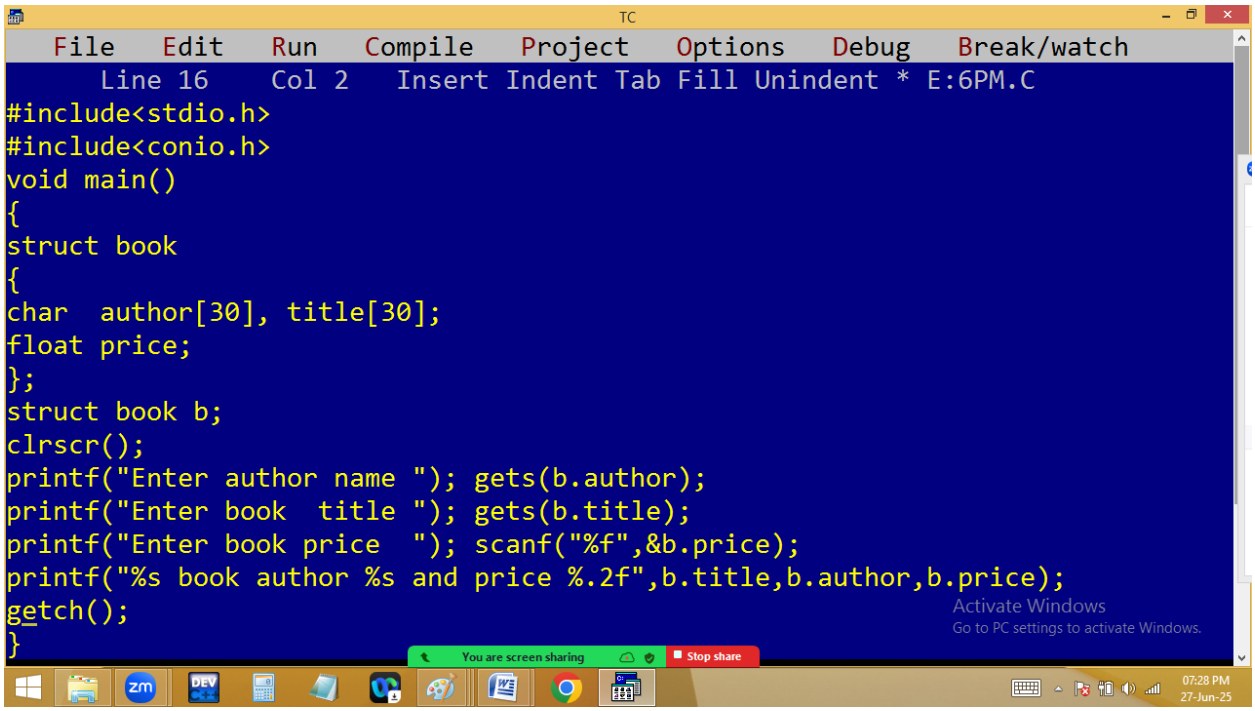

TC
Id=0, Name=, Fee=0.00
Id=-5120, Name=♦, Fee=16388970590970493000000.00
Activate Windows
Go to PC settings to activate Windows.
You are screen sharing
Stop share
07:20 PM
27-Jun-25

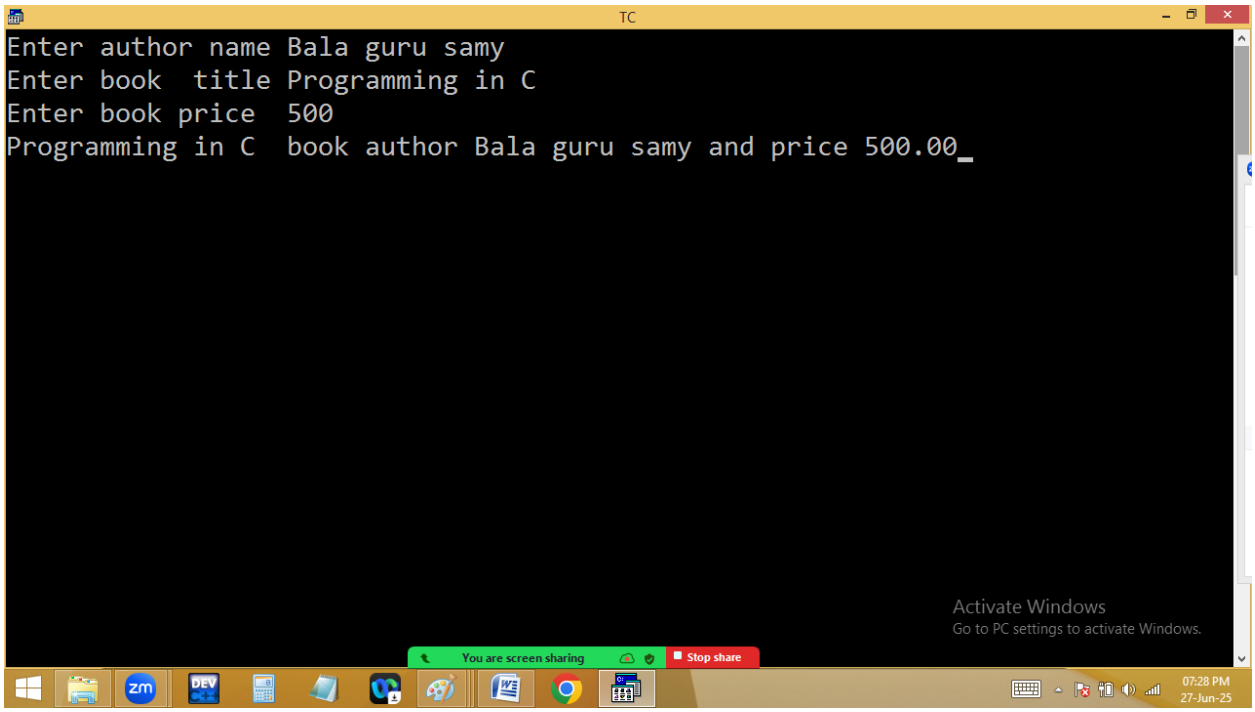
Finding structure size:





Reading and printing structure data:


TC
File Edit Run Compile Project Options Debug Break/watch
Line 16 Col 2 Insert Indent Tab Fill Unindent * E:6PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
struct book
{
char author[30], title[30];
float price;
};
struct book b;
clrscr();
printf("Enter author name "); gets(b.author);
printf("Enter book title "); gets(b.title);
printf("Enter book price "); scanf("%f",&b.price);
printf("%s book author %s and price %.2f",b.title,b.author,b.price);
getch();
}
Activate Windows
Go to PC settings to activate Windows.
You are screen sharing
Stop share
07:28 PM
27-Jun-25


TC
Enter author name Bala guru samy
Enter book title Programming in C
Enter book price 500
Programming in C book author Bala guru samy and price 500.00
Activate Windows
Go to PC settings to activate Windows.
You are screen sharing
Stop share
07:28 PM
27-Jun-25


TC
Enter author name Yashwant Kanitkar
Enter book title Let Us C
Enter book price 600
Let Us C book author Yashwant Kanitkar and price 600.00
Activate Windows
Go to PC settings to activate Windows.
You are screen sharing
Stop share
07:29 PM
27-Jun-25